



I am generating them on the FastAPI backend using Python

Because you are generating these reports directly from your FastAPI Python backend, you have a massive advantage: you do not need to fight with messy browser CSS or unreliable print dialogues to get perfect document formatting.

To achieve the best possible legibility and structure for the "Full Narrative" export, here is the best-practice architecture for generating Word (.docx) and PDF files in FastAPI.

1. Document Generation Strategy: Use Templates

Do not attempt to programmatically build the document node-by-node (e.g., `doc.add_heading(...)`) in your API routes. This couples your business logic tightly to your presentation layer and makes it very hard to tweak the design.

For Word (.docx): Use `python-docx-template` (`docxtpl`)

`docxtpl` is an incredible library that allows you to design your report template *directly inside Microsoft Word*.^{[1] [2]}

1. Open Word, set up your 1.5 line spacing, ragged-right alignment, colors, and headers perfectly.
2. Insert Jinja2-style tags directly into the Word document (e.g., `{{ executive_summary }}`), `{% for block in narrative_blocks %}`).
3. Save it as `template.docx`.
4. In FastAPI, pass the LangGraph output dictionary to `docxtpl`, and it generates a perfectly formatted .docx file instantly.^[1]

For PDF: Use `WeasyPrint` with HTML/Jinja2

If you want to offer PDF downloads, HTML-to-PDF is the most reliable approach in Python.^{[3] [4]}

1. Create an HTML template using Jinja2 that enforces strict print-CSS rules (e.g., `@page { margin: 1in; }`).
2. Pass your AI narrative data into the Jinja template.
3. Use the `WeasyPrint` library in FastAPI to convert that rendered HTML string directly into a byte stream for PDF download.^{[5] [3]}

2. Structuring the Payload Before Templating

Before you pass data from LangGraph to your template engine, shape it programmatically to ensure it meets the best practices we discussed.

Your Pydantic models in FastAPI should look something like this:

```
from pydantic import BaseModel
from typing import List

class NarrativeBlock(BaseModel):
    block_id: str
    timestamp: str
    metadata_tags: str # e.g., "Location: The Bridge | Sentiment: High Tension"
    summary: str       # The 1-sentence TL;DR
    content: str       # The dense text with line breaks

class FullNarrativeReport(BaseModel):
    executive_summary: str
    blocks: List[NarrativeBlock]
```

3. FastAPI Implementation Best Practices

Since rendering Word documents and converting HTML to PDFs are CPU-bound tasks, doing them synchronously will block your FastAPI async event loop, causing other users' requests to hang.^[1]

- **Streaming Responses:** Use `StreamingResponse` from `fastapi.responses` combined with an in-memory `BytesIO` buffer. This prevents you from having to save physical files to your server's disk just to send them to the user.^[1]
- **Thread Pooling:** If your narratives are extremely long (e.g., hundreds of pages), wrap the `template.render()` or `weasyprint.HTML().write_pdf()` calls in `starlette.concurrency.run_in_threadpool`. This pushes the heavy CPU rendering to a separate thread, keeping your FastAPI app lightning fast.^{[5] [1]}

Example FastAPI Implementation (docxtpl):

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from docxtpl import DocxTemplate
from io import BytesIO
import asyncio

app = FastAPI()

@app.post("/api/export/docx")
async def export_narrative_docx(report_data: FullNarrativeReport):
    # Load your perfectly formatted Word template
    template = DocxTemplate("templates/narrative_template.docx")

    # Render the template with the data dict (run in threadpool to avoid blocking)
```

```

await asyncio.to_thread(template.render, report_data.dict())

# Save to an in-memory buffer
file_stream = BytesIO()
template.save(file_stream)
file_stream.seek(0)

# Stream it directly to the user's browser for download
return StreamingResponse(
    file_stream,
    media_type="application/vnd.openxmlformats-officedocument.wordprocessingml.doc",
    headers={"Content-Disposition": "attachment; filename=Diamond_Miner_Narrative.d"}
)

```

By keeping the styling strictly inside a template file and passing cleanly structured LangGraph data via FastAPI, you guarantee that every downloaded report is professional, highly legible, and easy to maintain.

✱

1. <https://dev.to/huseinkntrc/automating-word-document-creation-with-python-and-fastapi-using-python-docx-template-41l>
2. <https://docxtpl.readthedocs.io>
3. <https://davidmuraya.com/blog/fastapi-create-secure-pdf/>
4. <https://www.promptzone.com/resumeburger/automate-your-resume-pdf-generation-with-ai-a-fastapi-weasyprint-tutorial-k5e>
5. <https://stackoverflow.com/questions/68832433/fonts-in-pdf-exported-using-weasyprint-inside-docker-fastapi>
6. <https://auth0.com/blog/fastapi-best-practices/>
7. <https://github.com/zhanymkanov/fastapi-best-practices>
8. <https://baysconsulting.co.uk/generating-word-documents-using-a-template-in-python/>
9. <https://www.youtube.com/watch?v=uVPc9As7Kkw>
10. https://www.reddit.com/r/Python/comments/wrt7om/fastapi_best_practices/